ControlPlane

Team Nexus, IIT Jodhpur — Accenture Innovation Challenge 2026, Problem Track 1

A verification layer between an application and any LLM API. It scores every response for hallucination, bias, privacy and cost, then decides whether to pass, annotate, redact, regenerate, escalate or block it — according to the risk policy of the use case the request came from.

Deliverable	Where
Business proposal	`docs/ControlPlane_Business_Proposal.pdf` · `.pptx`
Benchmark and results	`eval/results/report.md`
Prototype	this repository
Demo video	Watch on Google Drive

Run it

1. Install

```
git clone https://github.com/m25csa003-glitch/controlplane.git
cd controlplane
python3 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Takes a few minutes — torch and transformers are large.

2. Run the demo

```
python3 demo/run_pipeline.py
```

That is the whole smoke test, and it works with **no API key and no models downloaded**. Grounding falls back to a lexical check, the judge to an offline one, and the output says which mode produced it. Nothing here only works on the author's laptop.

You should see the same response taking a different verdict under each policy, and `audit chain intact: True` at the end.

3. Make it real (optional)

```
python3 load_models.py               # tier 1 classifiers, ~500 MB, one time
cp .env.example .env                 # then put OPENAI_API_KEY or ANTHROPIC_API_KEY in it
python3 demo/run_pipeline.py --models
```

The judge uses whichever key is present. No code change to switch provider.

What else you can run

```
python3 demo/run_pipeline.py --models --compare   # one input, three verdicts side by side
python3 demo/run_streaming.py --models            # verdict ready when the last token is
python3 demo/run_judge.py --live                  # what tier 2 is for, with its reasoning
python3 demo/run_feedback.py                      # reviews retuning thresholds
python3 -m pytest tests/ -q                       # 58 tests, no network
```

The gateway and dashboard

Two terminals.

```
# terminal 1 — takes ~7s to start while it loads the classifiers
uvicorn controlplane.proxy.gateway:app --port 8000

# terminal 2 — synthetic traffic so the dashboard has something to show
python3 demo/feed_dashboard.py
```

Then open **<http://localhost:8000/dashboard>**.

One HTML file served by FastAPI over an SSE feed — no build step, no node. It shows the cascade as a funnel, the live decision feed, the review queue, and each use case measured against **its own** declared limits: escalation rate against that policy's cap, p95 latency against that policy's budget. Both turn red when a policy exceeds a bound it set for itself.

Stop both with `Ctrl-C`, or `pkill -f uvicorn`.

The benchmark

Results are already committed. Re-running costs about a dollar and 45 minutes:

```
python3 eval/run_eval.py --lexical      # free: no models, no API
python3 eval/run_eval.py                # the real one
python3 eval/run_eval.py --report-only  # rewrite the report from the last run
```

`docs/commands.md` **lists every command and what it costs**, including how to point the gateway at a real provider.

How it works

A response passes through a cascade. Each tier runs only when the one before it was not confident enough.

Tier	What	Cost	Measured latency
0	Deterministic rules: PII, access control, schema	free	< 1 ms
1	NLI grounding, toxicity, bias heuristic	GPU seconds	87–190 ms p95
2	LLM judge, only inside the uncertainty band	tokens	1.3–3.1 s

The **action router** weighs `P(wrong) × cost_of_being_wrong` against `cost_of_human_review` and picks the cheaper mistake. Every category is evaluated, not just the first one that trips — a response can be a hallucination *and* a privacy event, and the verdict records both.

Every verdict, and every human override, goes to an append-only hash-chained log.

Why a policy layer

The same response, three policies, three verdicts:

```
hallucinated number  customer_support  -> regenerate   (being wrong costs Rs 400)
                     internal_copilot  -> annotate     (being wrong costs Rs 60)
                     decision_support  -> escalate     (being wrong costs Rs 50,000)
```

Nothing in the Python knows what a use case is. Risk appetite, latency budget, uncertainty band, thresholds, actions, retention, review capacity and jurisdiction all live in `configs/policies/*.yaml`. A new jurisdiction is a

config change.

Measured results

319 labelled cases, live judge (`openai/gpt-5.6-sol`); one call in 328 fell back, and the report says so — `eval/results/report.md`.

config	catch	false positives	tier 2 rate	p95	cost
rules only	22.8%	0.0%	0%	0.02 ms	Rs 0
rules + classifiers	93.5%	7.4%	0%	140 ms	Rs 0.35
the cascade	**94.6%**	**7.4%**	**2.8%**	**175 ms**	**Rs 1.10**
a judge on every response	92.4%	14.8%	100%	5534 ms	Rs 30.01

The cascade catches more than judging everything — 94.6% against 92.4% — at 3.7% of the cost, with half the false positives and 32× lower latency. That is the whole argument, and it is measured against a live judge rather than asserted.

The report also states what the system cannot do: multi-hop grounding at 3 of 9, quantifier flips at 8 of 15, two policies over their own latency budgets. Those are in `docs/tasks.md` under known gaps.

Layout

```
configs/policies/          one YAML per use case — this is where behaviour lives
configs/pricing.yaml       token prices, with source and capture date
controlplane/schema.py     the contract between modules
controlplane/tiers/        tier 0 rules, tier 1 classifiers, tier 2 judge
controlplane/router/       expected-cost action decision
controlplane/cost/         prices every check and every upstream call
controlplane/streaming.py  verification beside the token stream
controlplane/feedback/     reviews retuning thresholds
controlplane/audit/        hash-chained decision log
controlplane/proxy/        OpenAI-compatible gateway
controlplane/telemetry.py  in-memory view for the dashboard
dashboard/index.html       operator UI, one file, no build step
eval/                      benchmark harness, dataset and results
demo/                      runnable scenarios
docs/                      proposal, assumptions, status, commands
tests/                     58 tests, hermetic
```

Honest notes

- The eval set is synthetic and was written by the same person who tuned the checker. An earlier version scored 100% on everything, which is why the adversarial cases exist.
- Every rupee figure rests on assumed costs, listed in `docs/assumptions.md`.
- Grounding checks the answer against the retrieved sources, not the sources against the world. Bad retrieval still produces a confidently grounded wrong answer.
- Multi-turn risk accumulation is named in the brief and is not built. It is in the roadmap, not in the code.

Troubleshooting

Symptom	Cause
`command not found: python3`	virtualenv not activated
`tier 1: {'mode': 'lexical'...}`	models not loaded — pass `--models`, or run `load_models.py`
`no gateway at http://localhost:8000`	the feeder was started before the gateway
`Address already in use`	an old gateway is running — `pkill -f uvicorn`
`[tier2] ... judge failed`	key missing or wrong — it falls back and keeps going

Nothing in that list stops the pipeline. Every failure path falls back to something that still works, and reports which mode it ended up in.

Team

Nexus — Akshat Jain, Aditya Pratap Singh, Arnesh Sanjeev Singh. Round 2 implementation by Akshat Jain.